Effective Local Search Algorithms for the Vehicle Routing Problem with General Time Window Constraints

Toshihide Ibaraki[*] *Mikio Kubo*[†] *Tomoyasu Masuda*[*] *Takeaki Uno*[‡] *Mutsunori Yagiura*[*]

[*] Department of Applied Mathematics and Physics
Graduate School of Informatics
Kyoto University, Kyoto 606-8501, Japan
Email: {ibaraki, masuda, yagiura}@amp.i.kyoto-u.ac.jp

[†] Logistics and Information Engineering
Tokyo University of Mercantile Marine
Tokyo 135-8533, Japan
Email: kubo@ipc.tosho-u.ac.jp

[‡] Algorithm Foundation Research
National Institute of Informatics
National Center of Sciences, 2-1-2 Hitotsubashi, Chiyoda-ku, Tokyo 101-8430, Japan
Email: uno@me.titech.ac.jp

1 Introduction

The vehicle routing problem with time windows (VRPTW) is the problem to minimize the sum of the distances traveled by a fixed number of vehicles, which visit every customer exactly once under capacity and time window constraints. This problem is a well-known combinatorial optimization problem and has a wide range of applications such as bank deliveries, postal deliveries, school bus routing and so on. Since it is known to be NP-hard, no exact algorithm for VRPTW exists unless P = NP, and a number of heuristics have been proposed in the literature.

The problem VRPTW usually allows only one time window for each customer [3, 2, 5]. In this paper, we allow a general time window constraint (VRPGTW), in the sense that one or more time windows are allowed for each customer. In our formulation, the time window constraint for each customer is treated as a penalty function, which can be non-convex and discontinuous as long as it a piecewise linear function. Hence, after fixing the order of customers for a vehicle to visit, we must determine the optimal start times to serve the customers so that the total time penalty of the vehicle is minimized. We show that this problem can be efficiently solved by using dynamic programming. This dynamic programming is incorporated in the local search algorithms.

In our local search, in addition to standard neighborhoods, we use a new type of neighborhood called the cyclic exchange neighborhood, whose size generally grows exponentially with the input size. To overcome this difficulty, we propose an efficient heuristic to find an improved solution in the cyclic exchange neighborhood via the improvement graph. The computational results for various benchmark instances exhibit good prospects of the proposed algorithms.

2 Problem

Let $G = (V, E)$ be the complete directed graph with $V = \{0, 1, 2, \cdots, n\}$ and $E = \{(i, j) : i, j \in V, i \neq j\}$. In G, vertex 0 represents the depot and other vertices are customers to be serviced. Each vertex $i \in V$ is associated with: (a) the quantity q_i of goods to be delivered, (b) the penalty function $p_i(t)$ which is dependent on the start time t of the service, and (c) the service time u_i for unloading the goods, where $q_0 = 0$ and $u_0 = 0$ are assumed. Note that the penalty function $p_i(t)$, representing the time window constraint, can be any piecewise linear function, which may be non-convex and discontinuous. Let $M = \{1, \cdots, m\}$ be the set of all vehicles. Each vehicle k has its capacity Q_k and must depart from the depot after time w_0^r. Finally, an asymmetric distance matrix (d_{ij}) and an asymmetric traveling time matrix (t_{ij}) are given.

Let σ_k denote the route travelled by vehicle k; i.e., $\sigma_k(h)$ denotes the hth customer of σ_k and $\boldsymbol{\sigma} = (\sigma_1, \cdots, \sigma_m)$ denote a set of vehicle routes. Then $\boldsymbol{\sigma}$ must satisfy the following two constraints: (1) each route starts and terminates at the depot, (2) each customer is visited exactly once. The objective is to find a $\boldsymbol{\sigma}$ that minimizes

$$cost(\boldsymbol{\sigma}) = D(\boldsymbol{\sigma}) + P(\boldsymbol{\sigma}) + Q(\boldsymbol{\sigma}), \tag{1}$$

where $D(\boldsymbol{\sigma})$ is the total distance traveled by all vehicles, $P(\boldsymbol{\sigma})$ is the total penalty for start times of services and $Q(\boldsymbol{\sigma})$ is the total excess amount of capacities.

3 Local search

Local search (LS) starts with an initial solution $\boldsymbol{\sigma}$, and repeatedly replaces the current solution $\boldsymbol{\sigma}$ with a better solution $\boldsymbol{\sigma}'$ in its neighborhood $N(\boldsymbol{\sigma})$ until no better solution exits in $N(\boldsymbol{\sigma})$, where neighborhood $N(\boldsymbol{\sigma})$ is a set of solutions obtainable by slightly perturbing the current solution $\boldsymbol{\sigma}$.

3.1 Neighborhoods

In our local search, the following four neighborhoods are used. For the following discussion, we define the length of a path to be the number of customers in the path.

(1) The intra-route exchange neighborhood is used to improve a single route, and is defined to be the set of solutions obtainable by removing a path whose length is at most $L_{\text{path}}^{\text{intra}}$ (a parameter) and reinserting it into another position of the same route, where the inserting position is limited to $L_{\text{ins}}^{\text{intra}}$ (a parameter) closest ones from the current position.

(2) The 2-opt* neighborhood [3] is a variant of the 2-opt neighborhood, and is defined to be the set of solutions obtainable by removing two edges from two different routes (one from each) and adding two different edges to reconnect the subroutes into two routes. In the resulting routes, the visiting order of customers in the subroutes is maintained, which is considered a merit of this neighborhood.

(3) The cross exchange neighborhood [2] is defined to be the set of solutions obtainable by exchanging two paths of length at most L^{cross} (a parameter), in two routes. This neighborhood can be regarded as a subset of the 4-opt neighborhood.

(4) The cyclic exchange neighborhood [1] is applicable to a wide variety of problems, and whose neighborhood size grows exponentially with the input size. This neighborhood is defined to be the set of solutions obtainable by cyclically exchanging two or more paths whose lengths are at most L^{cyclic} (a parameter), respectively. The number of paths to be exchanged is at most Ψ (a parameter) and all paths belong to different routes. This neighborhood includes the cross exchange neighborhood as a special case if $L^{\text{cyclic}} \geq L^{\text{cross}}$.

We refer the first three neighborhoods (1), (2) and (3) as *standard neighborhoods*, since they are widely used in the literature. As stated above, the size of the cyclic exchange neighborhood grows exponentially with n and Ψ. Therefore, enumerating all solutions in the neighborhood to identify an improved solution is computationally infeasible. However, the concept of the *improvement graph* can be utilized to implicitly search the neighborhood. We construct the improvement graph so that a negative cycle with a certain property in the graph represents an improved solution in the cyclic exchange neighborhood. We develop an efficient heuristic to find a desired negative cycle satisfying the property, since finding it is known to be NP-hard in general. Refer [4] for details of the construction of the improvement graph and the heuristic.

3.2 The dynamic programming algorithm to compute the optimal start times of services

When evaluating a new solution $\boldsymbol{\sigma}$ in the standard neighborhoods, $D(\boldsymbol{\sigma})$ and $Q(\boldsymbol{\sigma})$ in (1) are easily determined once the visiting order of customers in each route is given. However, we need to compute the start times of services of customers so that $P(\boldsymbol{\sigma})$ is minimized. In this section, we introduce the dynamic programming algorithm to compute start times for a given route σ_k. The same idea is also used in constructing the improvement graph for the cyclic exchange neighborhood.

Let n_k denote the number of customers in σ_k, $\delta^{(i)}$ be the number of pieces in the piecewise linear function $p_i(t)$, and let $\delta_k = \sum_{h=1}^{n_k} \delta^{(\sigma_k(h))} + \delta^{(0)}$. As stated before, $\sigma_k(h)$ denotes the hth customer served by the vehicle k. For convenience, we define $\sigma_k(0) = 0$ and $\sigma_k(n_k + 1) = 0$. Let $p_h^k(t)$ be the penalty function for the hth customer in σ_k, and τ_{h-1}^k be the traveling time from the $(h-1)$st customer to the hth customer. We define $f_h^k(t)$ to be the minimum sum of penalty values for customers $\sigma_k(0), \sigma_k(1), \ldots, \sigma_k(h)$ under the condition that all of them are served before time t. Then $f_h^k(t)$ can be computed by

$$\begin{array}{lll} f_0^k(t) = & \left\{ \begin{array}{l} +\infty, \\ 0, \end{array} \right. & \begin{array}{l} t \in (-\infty, w_0^r) \\ t \in [w_0^r, +\infty) \end{array} \\ f_h^k(t) = & \min\limits_{t' \le t} \left(f_{h-1}^k(t' - \tau_{h-1}^k) + p_h^k(t') \right), & 1 \le h \le n_k + 1. \end{array} \tag{2}$$

The minimum penalty value $P(\boldsymbol{\sigma})$ for the route can be obtained as $\min\limits_t f_{n_k+1}^k(t)$. Then the optimal start time $s_{\sigma_k(h)}$ of the service for each $h = 1, \ldots, n_k$ and the time r^k when the vehicle k returns to the depot can be computed by

$$\begin{array}{lll} r^k & = & \min \arg \min\limits_t f_{n_k+1}^k(t) \\ s_{\sigma_k(h)} & = & \min \arg \min\limits_{t \le s_{\sigma_k(h+1)} - \tau_h^k} f_h^k(t), \quad 1 \le h \le n_k, \end{array} \tag{3}$$

where $s_0 = r^k$ is assumed for convenience.

Let us consider the data structure for executing recursion (2). Since $p_h^k(t)$ is a piecewise linear function, $f_h^k(t)$ is also a piecewise linear function. Therefore we can store the functions that appear in recursion (2) in linked lists whose components store the information of pieces of the functions. More precisely, each component of the list store the interval and function values of the corresponding linear piece. The computation of $f_{h-1}^k(t' - \tau_{h-1}^k) + p_h^k(t')$ and $f_h^k(t)$ from $f_{h-1}^k(t)$ and $p_h^k(t)$ can be achieved in $O(\delta_k)$ time, since the maximum number of pieces in both $f_{h-1}^k(t)$ and $p_h^k(t)$ is $O(\delta_k)$. The computation of the total time penalty $P(\boldsymbol{\sigma})$ and the optimal start times of services by (3) can also be accomplished in $O(\delta_k)$ time, since it requires simply to scan the list of $f_h^k(t)$. Therefore, the time complexity of the algorithm is $O(n_k \delta_k)$. We can further reduce this computational time to $O(\delta_k)$ by exploiting the information from the previous search, though the details of this speed-up is omitted due to space limitation.

3.3 The incumbent solution

In our formulation of VRPGTW, we treat capacity and time window constraints as soft constraints. Therefore, a locally optimal solution output by LS may be infeasible if we regard the two constraints as hard ones. However, during the search, the algorithm may find the solutions satisfying both capacity and time window constraints. Some applications prefer such feasible solutions to locally optimal solutions. Thus, in addition to the objective function $cost(\boldsymbol{\sigma})$, we also allow the algorithm to have another criterion, $besteval(\boldsymbol{\sigma})$, for evaluating feasible solutions $\boldsymbol{\sigma}$, where $besteval(\boldsymbol{\sigma})$ should be designed considering user's requirement. In our algorithm, the incumbent solution is defined to be the best feasible solution with respect to *besteval* found during the search.

4 Metaheuristics

Among many possible metaheuristic algorithms, we use the iterated local search (ILS) and the adaptive multi-start local search (AMLS) to optimize VRPGTW. ILS repeatedly applies LS to a number of initial solutions which are generated by perturbing good solutions obtained in the previous search. AMLS is similar to ILS. It keeps a set P of good solutions found in the search, and generates initial solutions by combining the parts of the solutions in P.

5 Computational results

We conducted computational experiments on Solomon's benchmark instances to evaluate the effectiveness of the proposed algorithms. The algorithms were coded in C and run on an IBM IntelliStation (550 MHz, 384 MB memory).

Solomon's benchmark instances treat both capacity and time window constraints as hard constraints, and there is one time window $[w_i^r, w_i^d]$ associated with each customer i. In order to formulate these problems, we use the following function as penalty function $p_i(t)$ for customer i:

$$p_i(t) = \begin{cases} g(w_i^r - t), & t < w_i^r \\ 0, & w_i^r \le t \le w_i^d \\ g(t - w_i^d), & w_i^d < t, \end{cases}$$

where $g(> 0)$ is a parameter. We then use the *lexicographical* order of the vector $(Q(\boldsymbol{\sigma}), P(\boldsymbol{\sigma}), D(\boldsymbol{\sigma}))$ as $besteval(\boldsymbol{\sigma})$ discussed in Section 3.3, that is, we first minimize $Q(\boldsymbol{\sigma})$, then $P(\boldsymbol{\sigma})$, and finally $D(\boldsymbol{\sigma})$ in updating the incumbent solution.

The best solutions produced by ILS and AMLS, computational times in seconds and the best published solutions are reported in Table 1. Note that the number of vehicles for each instance is set to the one with which the best published solution is produced for the instance. In both ILS and AMLS, the local search is repeated 300 times. The figures in each row represent the distance $D(\boldsymbol{\sigma})$ of the best solution $\boldsymbol{\sigma}$ obtained by the algorithm and the computational time, respectively. Feasible solutions $\boldsymbol{\sigma}$ to VRPHTW, i.e., $P(\boldsymbol{\sigma}) = 0$ and $Q(\boldsymbol{\sigma}) = 0$ were obtained for all instances except those with a dagger (†) mark. A single asterisk ($*$) indicates a tie with the best published solution and a double asterisk ($**$) indicates that the best published solution has been improved. We obtained 4 improved solutions with 20 tie solutions among 48 instances presented in the table. These results are significant, since our algorithms can handle more general problems and not specially tailored to solve VRPHTW.

Table 1: The best solutions produced by ILS and AMLS, CPU time in seconds and the best published solutions.

type	ILS		AMLS		best
	distance	time	distance	time	
r101	* 1650.80	7463	* 1650.80	7515	1650.80
r102	1487.67	6866	* 1486.12	7281	1486.12
r103	1293.05	6261	1356.61	5059	1292.85
r104	988.85	6128	986.03	4725	982.01
r105	1389.43	7251	* 1377.11	6827	1377.11
r106	1260.49	7386	1260.48	6398	1252.03
r107	** 1111.44	6930	1121.93	6327	1113.69
r108	986.62	6160	984.51	6276	964.38
c101	* 828.94	5643	* 828.94	5223	828.94
c102	* 828.94	5686	* 828.94	5968	828.94
c103	* 828.06	6110	* 828.06	5920	828.06
c104	* 824.78	5182	* 824.78	4922	824.78
c105	* 828.94	6125	* 828.94	6352	828.94
c106	* 828.94	4386	* 828.94	4126	828.94
c107	* 828.94	4309	* 828.94	4239	828.94
c108	* 828.94	6610	* 828.94	6321	828.94
rc101	* 1696.95	7221	* 1696.95	7558	1696.94
rc102	1558.07	6472	†	6855	1554.75
rc103	1268.09	6077	1269.96	5341	1262.02
rc104	1140.31	5817	1138.86	4585	1135.48
rc105	1646.26	6906	†	6836	1633.72
rc106	1433.99	5895	†	5270	1427.13
rc107	1232.76	4896	1233.21	4074	1230.54
rc108	1153.66	5712	1191.09	4902	1139.82

type	ILS		AMLS		best
	distance	time	distance	time	
r201	1261.58	5853	1256.11	3984	1252.37
r202	1206.10	7869	1197.43	4419	1191.70
r203	953.68	8119	948.09	5300	942.64
r204	860.18	8875	** 842.74	4445	849.62
r205	1003.82	8339	1010.73	6138	994.42
r206	913.18	7946	931.80	5748	912.97
r207	916.20	7145	915.80	5332	914.39
r208	747.09	9453	745.25	5419	731.23
c201	* 591.56	5723	* 591.56	5292	591.56
c202	* 591.56	6686	* 591.56	6228	591.56
c203	* 591.17	7853	* 591.17	7323	591.17
c204	* 591.60	7109	* 591.60	6990	591.60
c205	* 591.88	6712	* 591.88	6645	591.88
c206	* 591.49	5923	* 591.49	5379	591.49
c207	* 591.29	6390	* 591.29	6345	591.29
c208	* 591.32	7160	* 591.32	6762	591.32
rc201	1424.75	7407	1451.41	5991	1406.94
rc202	1488.41	7278	†	4195	1389.57
rc203	1073.65	8292	1083.45	5631	1060.45
rc204	807.31	6141	808.88	4644	799.12
rc205	** 1301.16	5005	** 1300.90	3659	1302.42
rc206	1156.53	7348	** 1152.03	5871	1153.93
rc207	1071.25	8872	1072.59	6166	1062.05
rc208	833.97	6989	847.04	4882	829.69

**: improved solution
*: tie solution
†: infeasible solution

6 Conclusion

This paper introduced the vehicle routing problem with general time window constraints, VRPGTW. To solve VRPGTW, we proposed the dynamic programming algorithm to efficiently determine the optimal start times of services for customers in a given route. We also applied a new type of neighborhood, the cyclic exchange neighborhood, to enhance the power of the local search. The computational results on Solomon's benchmark instances indicate that the proposed algorithms are competitive to existing heuristic algorithms in the literature, even if our algorithms can handle more general time windows.

We also conducted computational experiments on the instances from the parallel machine scheduling problems with various time penalty functions, though the results [4] are omitted due to space limitation. The results exhibited good prospects of the proposed algorithms.

References

[1] R. K. Ahuja and J. B. Orlin. Very large-scale neighborhood search. *Proc. 15th National Conference of the Australian Society for Operations Research Inc. ASOR Queensland Branch and ORSJ Hokkaido Chapter Joint Workshop on Operations Research from Theory to Real Life*, pages 33–48, 1999.

[2] E. Taillard P. Badeau M. Gendreau F. Guertin and J. Y. Potvin. A tabu search heuristic for the vehicle routing problem with soft time windows. *Transportation Science*, 31:170–186, 1997.

[3] J. Y. Potvin B. L. Garcia T. Kervahut and J. M. Rousseau. The vehicle routing problem with time windows. part 1: Tabu search. *INFORMS Journal on Computing*, 8:158–164, 1996.

[4] T. Masuda. Effective local search algorithms for the vehicle routing problem with general time window constraints. *Master Thesis, Department of Applied Mathematics and Physics, Graduate School of Informatics, Kyoto University, Kyoto 606-8501, Japan*, 2001.

[5] M. M. Solomon. The vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35:254–265, 1987.